Project Intelligence Report

Vine Investment Research

---

1. System Overview

这是一个面向“投资研究”场景的全栈 AI 系统：用户可以在统一聊天界面里完成三类任务，分别是股票分析、行业/主题深度研究，以及基于上传 PDF 的文档问答。系统前端负责对话、文档上传与展示，后端则通过 Google ADK 组织多个智能体，将用户意图路由到不同任务流水线，并把实时执行结果以 SSE 形式流式返回。它解决的核心问题不是“单次问答”，而是把投资研究中的不同工作模式拆成结构化责任单元，让“搜索、推理、规划、报告、RAG 问答”能够在一个系统里协作运行。

---

2. System Architecture

Tech Stack

- Language / Runtime: Python 3.11+，TypeScript，Node.js
- Framework:
  - Backend: FastAPI + Google ADK
  - Frontend: React + Vite
- Database / Storage:
  - Session state: `InMemorySessionService`
  - Document retrieval: ChromaDB PersistentClient
- Model Layer:
  - LLM: LiteLLM 封装的 DeepSeek
  - Embedding: `sentence-transformers` (`all-MiniLM-L6-v2`)
- External Services:
  - Tavily Search
  - PDF parsing via PyMuPDF
- Infrastructure:
  - 本地开发为主，`Makefile` 提供前后端启动命令

Architectural Pattern

- 前后端分离
- 后端为“API Gateway + 多智能体编排层 + 本地 RAG 子系统”
- ADK 侧采用“顶层路由 Agent + 多条专用任务流水线”的分层模式

Layer Diagram

```
[User / Browser]
      ↓
[Frontend — React / Vite]
      ↓
[HTTP + SSE — /api/chat, /api/corpus/*]
```

```
      ↓
[FastAPI Gateway — backend/main.py]
      ↓
[ADK Runner + root_agent]
      ↓
[Stock Pipeline] [Deep Research Pipeline] [Doc QA Pipeline]
      ↓
[Tavily Search] [ChromaDB RAG] [Session State]
```

Entry Points

- Frontend entry: `frontend/src/main.tsx`
- Frontend shell: `frontend/src/App.tsx`
- Backend entry: `backend/main.py`
- Agent system entry: `backend/app/agent.py`

One-paragraph system behavior

- 浏览器端所有聊天请求统一发到 `/api/chat`；FastAPI 使用 `Runner(root_agent)` 启动 ADK 执行；`root_agent` 决定请求走股票分析、深度研究还是文档问答；结果通过 SSE 持续回推到前端。文档上传则走 `/api/corpus/upload`，经 PDF 解析、切块、嵌入后写入 ChromaDB，后续由 `doc_qa_agent` 使用 RAG 检索调用。

---

3. Core Modules

Module: `backend/main.py`

- **Role**: API Gateway / Config & Bootstrap
- **Why core**:
  - 是后端 HTTP 入口，承接所有外部请求
  - 直接初始化 `FastAPI`、`InMemorySessionService`、`Runner`
  - 负责把 ADK 事件转换为 SSE 输出
- **Summary**: 它是系统的后端接待台，负责接收前端请求、管理会话、连接智能体运行时，并把结果流式送回浏览器。

Module: `backend/app/agent.py`

- **Role**: Orchestrator
- **Why core**:
  - 顶层 agent，直接挂载 3 个下游任务系统
  - 决定所有用户输入的任务归属
  - 包含产品层面的路由规则，而不是单一业务逻辑
- **Evidence**:
  - 调用 3 个下游 agent：`stock_coordinator`、`deep_research_pipeline`、`doc_qa_agent`
  - 被后端入口 `backend/main.py` 直接引用
- **Summary**: 它是整个 AI 系统的“总调度员”，负责识别用户到底是在做股票分析、主题研究，还是文档问答。

Module: `backend/app/agents/stock_research/agent.py`

- **Role**: Orchestrator
- **Why core**:
  - 控制股票分析主流程
  - 编排 3 个专用分析 agent
  - 产出最终汇总结果 `stock_research_output`
- **Evidence**:
  - 下游调用 3 个子 agent：`data_analyst_agent`、`strategy_analyst_agent`、`risk_analyst_agent`
  - 对应完整业务链：数据分析 -> 策略建议 -> 风险评估
- **Summary**: 它把“研究一只股票”拆成连续三步，让数据、策略和风险分别由独立角色负责。

Module: `backend/app/agents/deep_research/agent.py`

- **Role**: Orchestrator
- **Why core**:
  - 使用 `SequentialAgent` + `LoopAgent` 搭出系统中最复杂的研究流水线
  - 同时包含计划阶段、搜索阶段、质检阶段、报告生成阶段
  - 依赖自定义 `escalation_checker` 来控制循环退出
- **Evidence**:
  - 顺序链：`planner_agent -> research_loop -> composer_agent`
  - 循环链：`researcher_agent -> critic_agent -> escalation_checker`
- **Summary**: 它是“先规划、再迭代研究、最后成文”的深度研究引擎，是整个系统里最接近研究助理的部分。

Module: `backend/app/agents/doc_qa/agent.py`

- **Role**: Orchestrator / RAG Consumer
- **Why core**:
  - 是文档问答能力的唯一执行入口
  - 直接使用 `rag_tool`
  - 将文档语料能力暴露给统一聊天系统
- **Summary**: 它负责把“上传文档”变成“可问答的知识库”。

Module: `backend/app/tools/tavily_search.py`

- **Role**: External Tool Adapter
- **Why core**:
  - 被 3 个 agent 直接复用：`data_analyst`、`planner`、`researcher`
  - 承担系统内所有“实时外部搜索”职责
- **Summary**: 它是系统连接外部世界的搜索接口，为股票分析和深度研究提供实时资料。

Module: `backend/app/rag/document_manager.py`

- **Role**: Async-like Processor / Ingestion Layer
- **Why core**:
  - 负责 PDF 解析、切块、元数据生成、入库
  - 是从“原始文件”到“可检索语料”的转换器
- **Summary**: 它把用户上传的 PDF 变成可供 RAG 检索的语义片段。

Module: `backend/app/rag/vector_store.py`

- **Role**: Data Layer
- **Why core**:
  - 封装 ChromaDB 的 collection 初始化、查询、增加、删除、列出
  - 是文档问答链路的真实数据底座
- **Summary**: 它是系统的文档语义存储层，负责把文本块变成可召回的知识片段。

Module: `frontend/src/components/ChatView.tsx`

- **Role**: UI Orchestrator
- **Why core**:
  - 是用户与 AI 系统交互的统一入口
  - 负责发起聊天请求、消费 SSE、渲染消息流
  - 决定前端看到的“多 agent 输出”如何合并呈现
- **Summary**: 它是前端主战场，用户所有核心任务都从这里进入系统。

Module: `frontend/src/utils/api.ts`

- **Role**: API Gateway (Frontend-side)
- **Why core**:
  - 统一封装聊天流、文档上传、文档列表、文档删除、会话创建
  - 将前端组件从底层通信细节中解耦
- **Summary**: 它是前端与后端之间的通信适配层。

---

4. Key Request Flows

Flow: 股票分析

Step	Component	What it does
1	`frontend/src/components/ChatView.tsx`	用户输入如 “Analyze NVDA stock”，前端将文本提交给 `/api/chat`
2	`frontend/src/utils/api.ts`	使用 `fetch` 建立 POST 请求并读取 SSE 流
3	`backend/main.py`	创建/复用会话，调用 `Runner(root_agent)`
4	`backend/app/agent.py`	根 agent 判断请求属于股票研究
5	`backend/app/agents/stock_research/agent.py`	`stock_coordinator` 启动股票研究主链
6	`data_analyst_agent`	调用 Tavily 搜索公司新闻、指标、财报、行业信息，写入 `market_data_output`
7	`strategy_analyst_agent`	读取 `market_data_output`，生成策略，写入 `strategy_output`

Step	Component	What it does
8	`risk_analyst_agent`	读取 `market_data_output` + `strategy_output`，评估风险，写入 `risk_output`
9	`stock_coordinator`	汇总三段结果，形成最终股票分析响应
10	`backend/main.py`	把 agent 事件流式转发给前端
11	`ChatView.tsx`	将结果追加为助手消息

Flow: 深度研究报告

Step	Component	What it does
1	`ChatView.tsx`	用户输入一个研究主题
2	`backend/main.py`	将请求交给 `root_agent`
3	`backend/app/agent.py`	根 agent 将请求路由到 `deep_research_pipeline`
4	`planner_agent`	生成研究计划，写入 `research_plan`
5	`research_loop`	启动循环执行
6	`researcher_agent`	搜索资料并写入 `research_notes`
7	`critic_agent`	对研究质量做结构化评估，写入 `research_evaluation`
8	`escalation_checker`	读取 `research_evaluation`，若通过则停止循环，否则继续
9	`composer_agent`	将计划和研究笔记整合为 `final_report`
10	`backend/main.py`	把最终研究报告流式传回前端

Flow: 上传 PDF 并进行文档问答

Step	Component	What it does
1	`frontend/src/components/DocumentUpload.tsx`	用户拖拽或选择 PDF
2	`frontend/src/utils/api.ts`	调用 `/api/corpus/upload` 上传文件
3	`backend/main.py`	接收文件并调用 `ingest_pdf()`
4	`backend/app/rag/document_manager.py`	解析 PDF，按 chunk 切分，生成 `source/page/doc_id` 元数据
5	`backend/app/rag/vector_store.py`	把分块和元数据写入 ChromaDB
6	用户再次回到 `ChatView.tsx`	提问某份已上传文档里的内容
7	`root_agent`	判断这是文档问答任务
8	`doc_qa_agent`	调用 `rag_tool` 检索语料片段
9	`rag_retrieval.py` + `vector_store.py`	从 ChromaDB 召回文本片段与来源页码

Step	Component	What it does
10	`doc_qa_agent`	组织答案并返回给前端

---

5. Business Scenarios

Scenario: “研究一只股票是否值得继续跟踪”

当用户输入股票代码或公司名称时，系统不会直接给出一句话结论，而是把任务拆成三个岗位：先由数据分析员收集市场背景、新闻、财报和行业动态；再由策略分析员根据这些数据给出不同风格的操作思路；最后由风险分析员判断这些建议是否和用户的风险承受能力匹配。对用户而言，这不是单轮问答，而是一次结构化投研过程。

Scenario: “围绕某个主题生成一份完整研究报告”

当用户提出一个宽泛主题，例如“研究 AI 芯片市场”时，系统先尝试把问题拆成研究计划，再进入循环式研究：搜资料、检查资料是否充分、补搜缺口，直到研究质量过关，最后再汇总成正式报告。这个流程的价值在于，它不是直接让大模型“编一篇文章”，而是强制先做研究，再写结论。

Scenario: “把 PDF 变成可查询知识库”

用户上传年报、研报或论文之后，系统会在后端把 PDF 解析成文本片段并存入语义向量库。之后用户无需自己翻页找内容，只要自然语言提问，系统就能把相关段落召回出来，再结合页码和来源生成回答。这一能力尤其适合看财报、研究报告和材料类文档。

Scenario: “前端上看起来像一个聊天产品，但后端其实是多角色协作”

从用户视角看，系统只有一个聊天框；但从系统内部看，每种任务都被分配给不同 agent。前端只负责把对话上下文展示出来，真正的任务分工发生在 ADK 层。这让产品外观保持简单，而内部执行能逐步扩展。

---

6. Important Data Models

> 该项目没有传统 ORM / 业务数据库 schema，核心数据模型主要是运行时状态和向量库元数据。

Model: `session.state`

Fields (key fields only):

```
market_data_output
strategy_output
risk_output
stock_research_output
research_plan
research_notes
research_evaluation
final_report
doc_qa_answer
```

Role: 这是系统当前会话内的“中间工作台”，承接各个 agent 的阶段性输出。

Relationships:

```
root_agent           → session.state   (writes/reads via sub-agents)
stock pipeline       → session.state   (multi-step output passing)
deep research loop   → session.state   (iterative refinement)
doc_qa_agent         → session.state   (final answer)
```

Model: Chroma document chunk

Fields (key fields only):

```
id
text
metadata.source
metadata.page
metadata.chunk_index
metadata.total_pages
metadata.doc_id
distance
```

Role: 这是系统在产品层面的“文档知识原子单元”，每个 chunk 都代表上传 PDF 中的一小段可检索内容。

Relationships:

```
PDF file      → multiple chunks     (1:N)
chunk         → metadata            (1:1)
doc_qa_agent  → retrieved chunks    (N:N over queries)
```

Model: SSE event payload

Fields (key fields only):

```
author
type
text
state_update
```

Role: 这是后端传给前端的统一运行时消息格式，用来表示当前是哪位 agent 在输出、是否为最终响应，以及有哪些状态更新。

Relationships:

```
Runner event  → SSE event payload   (1:1 transformed)
SSE payload   → ChatView            (stream rendering)
```

---

7. Domain Concepts

Concept: `output_key`

- **Meaning**: 每个子 agent 完成某一步之后写入 session state 的键名。
- **Purpose**: 让后续 agent 能在不直接耦合代码调用的情况下读取上一阶段的结果，实现“通过状态接力”的任务编排。

Concept: `research_evaluation`

- **Meaning**: 深度研究流程中的质检结果，通常是带 `grade/comment/follow_up_queries` 的结构化输出。
- **Purpose**: 它决定研究循环是否继续，是深度研究链路的质量闸门。

Concept: `doc_id`

- **Meaning**: 上传 PDF 的文档级标识符，用于把多个 chunk 归属于同一份原始文件。
- **Purpose**: 用来支持文档删除、聚合展示、来源识别等文档管理操作。

Concept: `stock_research_output`

- **Meaning**: 股票研究流水线的顶层汇总结果。
- **Purpose**: 用于把数据分析、策略建议和风险评估收敛成一次对用户可消费的最终答复。

Concept: `research_plan`

- **Meaning**: 深度研究开始前生成的任务计划。
- **Purpose**: 它定义了“研究应该围绕哪些问题展开”，避免模型直接跳到报告生成，保证任务有结构。

---

8. Complexity Map

Level	Files	Reason
🔥 High	`backend/main.py`, `backend/app/agent.py`, `backend/app/agents/deep_research/agent.py`, `backend/app/agents/deep_research/sub_agents/escalation_checker.py`	处于核心执行链路；控制 HTTP、会话、任务路由、循环终止；修改后影响整条系统路径

Level	Files	Reason
🟡 Medium	`backend/app/agents/stock_research/agent.py`, `backend/app/agents/stock_research/prompt.py`, `backend/app/rag/document_manager.py`, `backend/app/rag/vector_store.py`, `frontend/src/components/ChatView.tsx`, `frontend/src/utils/api.ts`	含主要业务逻辑或对接关键基础设施，但边界较清晰
🟢 Low	`backend/app/agents/doc_qa/agent.py`, `frontend/src/components/ResearchPlanApproval.tsx`, `frontend/src/components/ActivityTimeline.tsx`, `frontend/src/components/DocumentUpload.tsx`	单一职责、局部功能明确，风险相对可控

特别注意

- `backend/main.py` 同时承担 API、SSE、Session Adapter 责任，耦合度偏高。
- `deep_research` 是系统中最复杂的 agent 编排区域，尤其是循环退出和状态读取。
- RAG 文档管理前后端有一处实现落差：后端 `list_documents()` 不返回 `doc_id`，但前端删除按钮依赖 `doc_id` 才显示。
- `ResearchPlanApproval.tsx` 与 `ActivityTimeline.tsx` 已实现但尚未接入主界面，说明产品意图已经存在，落地还未完成。

---

9. Developer Onboarding Path

Order	File / Area	Why
1	`backend/main.py`	看清系统如何启动、暴露哪些 API、如何把 Runner 接到 HTTP
2	`backend/app/agent.py`	理解顶层任务路由，是进入整个 agent 系统的总入口
3	`backend/app/agents/stock_research/`	这是最容易理解的一条多 agent 流水线，适合作为 ADK 编排入门
4	`backend/app/agents/deep_research/`	理解高级编排：Sequential + Loop + 自定义 BaseAgent
5	`backend/app/rag/` 和 `backend/app/tools/`	理解系统如何接外部搜索和本地 RAG
6	`frontend/src/utils/api.ts`	看清前后端协议，尤其是 SSE 聊天流
7	`frontend/src/components/ChatView.tsx`	理解 UI 如何消费多 agent 输出
8	`frontend/src/components/DocumentUpload.tsx`	理解文档摄取入口与当前实现缺口

First task suggestion

- 最适合作为第一个任务的是：把 `ResearchPlanApproval.tsx` 接入深度研究主流程，真正实现结构化的计划审批，而不是只在聊天文本里要求用户确认。这能触达核心体验，但不会直接破坏底层 RAG 或搜索逻辑。

Appendix: PM View

Core Features

- 股票分析：将单只股票研究拆成数据、策略、风险三步
- 深度研究：先规划、再搜集、再质检、最后生成正式报告
- 文档问答：上传 PDF 后以自然语言检索内容
- 流式对话：用户能在统一聊天界面中使用全部能力

Key Business Rules

- 每类用户任务都必须先经过 `root_agent` 路由
- 股票分析依赖顺序责任划分，而不是单个 agent 包打天下
- 深度研究必须先有计划，再允许进入研究循环
- 文档问答必须基于已上传语料，而不是纯模型记忆
- 运行时状态保存在内存 session 中，因此会话信息不是持久化的

User-Facing Flows

- Flow 1：用户打开聊天界面 -> 输入股票或研究问题 -> 系统自动选路 -> 分阶段返回结果
- Flow 2：用户进入 Documents -> 上传 PDF -> 回到聊天界面提问 -> 系统基于文档回答
- Flow 3：用户输入宽泛研究主题 -> 系统先生成计划 -> 迭代搜集资料 -> 输出完整研究报告

Notes for Future Maintainers

- 当前系统最强的部分不是单个 agent，而是“任务拆分 + 状态接力”。
- 真正决定产品体验的核心，不是模型本身，而是三条流水线的边界设计是否清晰。
- 如果未来要继续扩展，优先方向应是：
  - 接通 `ResearchPlanApproval.tsx`
  - 接通 `ActivityTimeline.tsx`
  - 让文档列表返回 `doc_id`，补齐删除链路
  - 将 `InMemorySessionService` 替换为持久化 SessionService